# Assignment 3 — C Programming Error Analysis

| | | | | |
|---|---|---|---|---|
| 1 | 8 | Missing semicolon after 'int x = 10' | Syntax | int x = 10; |
| 1 | 13 | Assignment used instead of comparison (x = 10) | Semantic | if(x == 10) |
| 1 | 17 | Assigning string literal to int variable | Semantic | int marks = 90; |
| 1 | 19 | Using undeclared variable 'z' | Semantic | Declare 'int z;' before use |
| 1 | 21 | Assignment to constant (100 = x) | Syntax | x = 100; |
| 1 | 25 | Array index out of bounds (arr[10] for arr[5]) | Runtime | Use index 0–4 only, e.g. arr[4] = 50; |
| 1 | 27 | Assigning float pointer to int variable without cast | Semantic | float *ptr = (float *)&x; or use correct type |
| 1 | 29 | Using %f format specifier for int variable x | Semantic | printf("%d\n", x); |
| 1 | 33 | Division by zero (a/b where b=0) | Runtime | Ensure b != 0 before dividing |
| 1 | 35 | Multi-character character constant 'AB' | Syntax | char ch = 'A'; |
| 1 | 39 | Logical error: both if and else print 'X is greater' | Logical | else printf("Y is greater"); |
| 1 | 42 | Return value of sqrt() not used / missing printf | Logical | printf("%f\n", sqrt(25)); |
| 1 | 44 | Calling undeclared function 'show()' | Semantic | Declare/define show() before calling |
| 1 | 46 | Semicolon after while condition causes infinite loop | Logical | Remove ';': while(x < 15) { |
| 1 | 49 | 'continue' used outside a loop | Syntax | Remove the continue; statement |
| 1 | 51 | 'break' used outside a loop/switch | Syntax | Remove the break; statement |
| 1 | 53 | Function called with 1 argument but requires 2 (calculate) | Semantic | int result = calculate(10, 20); |
| 2 | 9 | Missing semicolon after 'int num2 = 20' | Syntax | int num2 = 20; |
| 2 | 21 | Missing closing ')' in if(choice == 1 | Syntax | if(choice == 1) |
| 2 | 31 | Extra closing ')' in else if(choice == 3)) | Syntax | else if(choice == 3) |
| 2 | 43 | Missing closing ')' in for loop: for(int i=0; i<5; i++ | Syntax | for(int i=0; i<5; i++) |
| 2 | 49 | Missing closing ')' in while(choice > 0 | Syntax | while(choice > 0) |
| 2 | 54 | Missing semicolon after while(choice > 5) in do-while | Syntax | while(choice > 5); |
| 2 | 62 | Missing colon after 'case 2' | Syntax | case 2: |
| 2 | 77 | Unclosed character literal char grade = 'A; | Syntax | char grade = 'A'; |
| 2 | 80 | Missing ']' in array declaration int arr[5; | Syntax | int arr[5]; |
| 2 | 91 | Missing closing '}' for inner block (unmatched brace) | Syntax | Add '}' to close the block |
| 2 | 99 | Missing semicolon after last printf | Syntax | printf("End of Program\n"); |
| 3 | 8 | Assigning string literal to int variable 'age' | Semantic | int age = 20; |
| 3 | 12 | Assigning string literal to char variable 'grade' | Semantic | char grade = 'A'; |
| 3 | 14 | Using %d format for float variable 'salary' | Semantic | printf("%f\n", salary); |
| 3 | 16 | Using %f format for int variable 'marks' | Semantic | printf("%d\n", marks); |
| 3 | 20 | Using undeclared variable 'score' | Semantic | Declare 'int score;' before use |
| 3 | 23 | Re-declaration of variable 'x' | Semantic | Remove the second 'int x = 20;' |
| 3 | 25 | Assignment to constant (100 = x) | Syntax | x = 100; |
| 3 | 27 | Using %s format for int variable 'num' | Semantic | printf("%d\n", num); |
| 3 | 31 | Using %d format for float variable 'pi' | Semantic | printf("%f\n", pi); |
| 3 | 34 | Using float as array index (arr[2.5]) | Semantic | arr[2] = 100; |
| 3 | 36 | Assigning float 65.5 to char (loss of data) | Semantic | char ch = 'A'; // ASCII 65 |
| 3 | 39 | Calling add() with 1 argument instead of 2 | Semantic | result = add(10, 5); |
| 3 | 44 | Calling average() with 2 arguments instead of 3 | Semantic | avg = average(10, 20, 30); |
| 3 | 46 | Assigning string literal to int variable 'temperature' | Semantic | int temperature = 35; |
| 3 | 49 | Assigning string literal to float variable 'discount' | Semantic | float discount = 10.5; |
| 3 | 52 | Direct string assignment to char array (name = "Robil") | Semantic | strcpy(name, "Robil"); |
| 3 | 56 | Using undeclared variable 'totalMarks' | Semantic | Declare 'int totalMarks;' before use |
| 3 | 59 | Using %c format for int variable 'number' | Semantic | printf("%d\n", number); |

| | | | | |
|---|---|---|---|---|
| 3 | 62 | Using %d format for float variable 'rate' | Semantic | printf("%f\n", rate); |
| 3 | 66 | Calling add(a,b,c) with 3 args but function takes only 2 | Semantic | sum = add(a, b); |
| 3 | 73 | Assigning float result to int 'area' (precision loss) | Semantic | float area = 3.14 * radius * radius; |
| 3 | 78 | Assignment used instead of comparison (studentMarks = 100) | Semantic | if(studentMarks == 100) |
| 3 | 82 | Assigning string literal to int variable 'size' | Semantic | int size = 5; |
| 3 | 85 | Using %d format for float variable 'percentage' | Semantic | printf("%f\n", percentage); |
| 3 | 88 | Using string as array index (arr2["five"]) | Semantic | arr2[5] = 100; |
| 3 | 95 | Using string in case label (case "2":) | Syntax | case 2: |
| 3 | 100 | Calling undeclared function 'multiply()' | Semantic | Declare/define multiply() before calling |
| 3 | 106 | Using %d format for float variable 'areaRect' | Semantic | printf("%f\n", areaRect); |
| 3 | 113 | Assigning float return to int 'avgMarks' from average() | Semantic | float avgMarks = average(marks1, marks2, marks3); |
| 3 | 116 | Using undeclared variable 'value' | Semantic | Declare 'int value;' before use |
| 3 | 119 | Assigning string literal to int variable 'employeeId' | Semantic | int employeeId = 101; |
| 4 | 35 | Calling undeclared/undefined function 'generateReport()' | Semantic | Declare and define generateReport() function |
| 4 | 36 | Calling undeclared/undefined function 'displayEmployee()' | Semantic | Declare and define displayEmployee() function |
| 4 | 37 | Calling undeclared/undefined function 'processData()' | Semantic | Declare and define processData() function |
| 4 | 38 | Calling undeclared/undefined function 'saveRecord()' | Semantic | Declare and define saveRecord() function |
| 4 | 39 | Calling undeclared/undefined function 'loadRecord()' | Semantic | Declare and define loadRecord() function |
| 4 | 40 | Calling undeclared/undefined function 'printInvoice()' | Semantic | Declare and define printInvoice() function |
| 4 | 41 | Calling undeclared/undefined function 'calculateTax()' | Semantic | Declare and define calculateTax() function |
| 4 | 42 | Calling undeclared/undefined function 'validateUser()' | Semantic | Declare and define validateUser() function |
| 4 | 43 | Calling undeclared/undefined function 'sendNotification()' | Semantic | Declare and define sendNotification() function |
| 4 | 44 | Calling undeclared/undefined function 'generateCertificate()' | Semantic | Declare and define generateCertificate() function |
| 4 | 55 | Missing definition for declared function 'calculateSalary()' | Semantic | Define calculateSalary(float basic) { ... } |
| 4 | 55 | Missing definition for declared function 'printStudentDetails()' | Semantic | Define printStudentDetails() { ... } |
| 4 | 55 | Missing definition for declared function 'showResult()' | Semantic | Define showResult() { ... } |
| 4 | 55 | Missing definition for declared function 'calculateAverage()' | Semantic | Define calculateAverage(int a, int b, int c) { ... } |
| 4 | 13 | extern variable 'companyProfit' declared but never defined | Semantic | Add definition: float companyProfit = 0.0; somewhere |
| 5 | 12 | Division by zero at runtime (a/b where b=0) | Runtime | Check b != 0 before dividing |
| 5 | 16 | Dereferencing NULL pointer ptr1 | Runtime | Allocate memory: int *ptr1 = malloc(sizeof(int)); |
| 5 | 20 | Array out of bounds: arr[10] for arr[5] | Runtime | Use valid index 0–4 |
| 5 | 23 | Dereferencing uninitialized pointer ptr2 | Runtime | Initialize: int *ptr2 = malloc(sizeof(int)); |
| 5 | 27 | Buffer overflow: copying 'Programming' into char[5] | Runtime | char name[12]; or copy shorter string |
| 5 | 31 | Reading uninitialized pointer ptr3 | Runtime | Initialize ptr3 before dereferencing |
| 5 | 37 | Using NULL file pointer fp without NULL check | Runtime | if(fp == NULL) { perror("Error"); return 1; } |
| 5 | 43 | Buffer overflow: copying 'Computer Science'(16) into malloc(5) | Runtime | ptr4 = malloc(17); // allocate enough space |
| 5 | 48 | Using freed pointer ptr4 (use-after-free) | Runtime | Do not access ptr4 after free() |
| 5 | 53 | Double free of ptr5 | Runtime | Remove second free(ptr5); |
| 5 | 58 | Modulo by zero (x % y where y=0) | Runtime | Ensure y != 0 before using % |
| 5 | 62 | Buffer overflow: strcat appends 'WorldProgramming' to str1[10] | Runtime | Increase str1 size: char str1[30] = "Hello"; |
| 5 | 68 | Heap overflow: writing 10 ints into malloc(3 * sizeof(int)) | Runtime | malloc(10 * sizeof(int)) or loop to i<3 |

| | | | | |
|---|---|---|---|---|
| 5 | 72 | Matrix out of bounds: matrix[5][5] on matrix[3][3] | Runtime | Use valid indices 0–2 only |
| 5 | 75 | Dereferencing NULL pointer ptr6 | Runtime | Allocate memory before use |
| 5 | 79 | gets() is unsafe — can cause buffer overflow | Runtime | Use fgets(password, 8, stdin); instead |
| 5 | 84 | Use-after-free: writing to freed ptr7 | Runtime | Do not write to ptr7 after free() |
| 5 | 87 | Reading uninitialized variable 'age' | Runtime | Initialize: int age = 0; |
| 5 | 90 | Negative array index arr[-1] | Runtime | Use non-negative index only |
| 5 | 93 | Dereferencing uninitialized pointer ptr8 | Runtime | Initialize ptr8 before use |
| 5 | 96 | Printing NULL char pointer with %s | Runtime | Initialize: char *text = "some text"; |
| 5 | 100 | Possible NULL return from malloc not checked | Runtime | Check hugeArray != NULL after malloc |
| 5 | 105 | Infinite loop incrementing NULL pointer ptr9 | Runtime | Remove or fix the infinite loop |
| 6 | 12 | Wrong condition: num%2==1 means odd, not even | Logical | if(num % 2 == 0) printf("Number is Even"); |
| 6 | 22 | Grade logic inverted: marks>=90 prints Grade B (should be A) | Logical | if(marks >= 90) printf("Grade A\n"); // etc. |
| 6 | 34 | Leap year check incomplete (divisible by 100/400 missing) | Logical | if((year%4==0 && year%100!=0)\|\|(year%400==0)) |
| 6 | 42 | Voting eligibility: age>18 prints Not Eligible (should be Eligible) | Logical | if(age >= 18) printf("Eligible for Voting\n"); |
| 6 | 50 | Smallest number logic is inverted (uses a>b to find smaller) | Logical | if(a < b) printf("Smaller = %d", a); else printf("Smaller = %d", b); |
| 6 | 60 | Largest logic wrong: uses < to find largest (should use >) | Logical | if(y > largest) largest = y; if(z > largest) largest = z; |
| 6 | 68 | Integer division loses decimal for average | Logical | average = (float)totalMarks / subjects; |
| 6 | 74 | Circle area formula wrong: should be PI*r^2 not 2*PI*r^2 | Logical | area = 3.14 * radius * radius; |
| 6 | 81 | Bonus condition logic inverted (salary>30000 gives 2% instead of 20%) | Logical | if(salary <= 30000) bonus = salary*0.20; else bonus = salary*0.02; |
| 6 | 90 | Switch month cases off by one (case 1 prints February etc.) | Logical | case 1: printf("January"); case 2: printf("February"); etc. |
| 6 | 103 | Prime check sets isPrime=1 when divisor found (should be 0) | Logical | isPrime = 0; break; // inside if(n%i==0) |
| 6 | 109 | Discount is added instead of subtracted | Logical | finalAmount = purchaseAmount - discount; |
| 6 | 117 | Sum loop assigns i to sum instead of adding (sum = i) | Logical | sum += i; |
| 6 | 125 | Temperature logic inverted: <40 prints Cold (35 would be Hot) | Logical | if(temperature >= 40) printf("Hot Weather"); else printf("Cold Weather"); |
| 6 | 131 | Grade 'A' message inverted: prints 'Average' for A | Logical | if(grade == 'A') printf("Excellent Performance\n"); |
| 6 | 137 | Rectangle area uses + instead of * | Logical | rectangleArea = length * width; |
| 6 | 143 | Pass/Fail condition inverted: >=60 prints Fail | Logical | if(percentage >= 60) printf("Pass\n"); else printf("Fail\n"); |
| 7 | 8 | Array out of bounds in loop: i<=5 accesses marks[5] | Runtime | for(int i=0; i<5; i++) |
| 7 | 18 | Integer division for average (total/5 loses decimal) | Logical | average = (float)total / 5; |
| 7 | 27 | Loop index starts at 1 causing numbers[0] uninitialized and numbers[10] out of bounds | Runtime | for(int i=0; i<10; i++) numbers[i] = (i+1)*10; |
| 7 | 33 | Buffer overflow: 'ComputerScience'(15) into char name[10] | Runtime | char name[16]; or use shorter string |
| 7 | 40 | Direct string assignment to char array (city = "Delhi") | Semantic | strcpy(city, "Delhi"); |
| 7 | 47 | Comparing strings with == instead of strcmp() | Logical | if(strcmp(str1, str2) == 0) |
| 7 | 53 | gets() is unsafe — can cause buffer overflow | Runtime | fgets(password, 8, stdin); |
| 7 | 63 | grade[] not null-terminated, printf("%s") may print garbage | Runtime | grade[2] = '\0'; // null-terminate |
| 7 | 69 | Negative array index arr[-1] — undefined behavior | Runtime | Use valid index >= 0 |
| 7 | 76 | Buffer overflow: 'student@gmail.com'(17) into email[15] | Runtime | char email[20] = "student@gmail.com"; |
| 7 | 82 | scores[3] out of bounds for scores[3] (valid: 0–2) | Runtime | Remove scores[3]=60; and access only 0–2 |
| 7 | 86 | Buffer overflow: 'Programming'(11) into subject[10] | Runtime | char subject[12] = "Programming"; |
| 7 | 92 | Comparing string with == instead of strcmp() | Logical | if(strcmp(username, "admin") == 0) |

| | | | | |
|---|---|---|---|---|
| 7 | 98 | Buffer overflow: '987654321012'(12) into mobile[10] | Runtime | char mobile[13]; or use shorter number |
| 7 | 104 | Off-by-one: loop i<=5 accesses attendance[5] out of bounds | Runtime | for(int i=0; i<5; i++) |
| 7 | 110 | Out-of-bounds access: course[20] on char[20] (valid 0–19) | Runtime | printf("%c\n", course[19]); or course[0] |
| 7 | 116 | college[] not null-terminated — printf("%s") undefined | Runtime | college[5] = '\0'; |
| 7 | 126 | section[3] too small for 'IT-A'(4 chars + null) | Runtime | char section[5] = "IT-A"; |
| 8 | 15 | Calling add() with 1 argument instead of 2 | Semantic | result = add(num1, num2); |
| 8 | 20 | Calling average() with 2 arguments instead of 3 | Semantic | avg = average(70, 80, 90); |
| 8 | 33 | swap(x,y) passes by value — swap has no effect | Logical | Change swap to take pointers: swap(&x, &y); and update definition |
| 8 | 41 | Calling undeclared function 'calculateGrade()' | Semantic | Declare/define calculateGrade() before calling |
| 8 | 45 | Calling undeclared function 'circleArea()' | Semantic | Declare/define circleArea() before calling |
| 8 | 50 | Calling undeclared function 'sumArray()' | Semantic | Declare/define sumArray() before calling |
| 8 | 55 | Calling undeclared function 'displayResult()' | Semantic | Declare/define displayResult() before calling |
| 8 | 64 | factorial(0) returns 0 instead of 1 | Logical | if(n == 0) return 1; |
| 8 | 78 | maximum() returns wrong value (returns smaller number) | Logical | if(a > b) return a; else return b; |
| 8 | 87 | fibonacci() infinite recursion: fibonacci(n)+fibonacci(n-1) should be (n-1)+(n-2) | Logical | return fibonacci(n-1) + fibonacci(n-2); |
| 8 | 96 | simpleInterest() returns nothing (missing return value) | Logical | return si; |
| 8 | 107 | power() loop starts at i=1, calculates base^(exp-1) | Logical | for(int i=0; i<exp; i++) // start from 0 |
| 8 | 119 | reverseNumber() has no return statement | Logical | return rev; // at end of function |
| 8 | 139 | isPrime() returns 1 when divisor found (should return 0) | Logical | return 0; // not prime when divisible |
| 8 | 147 | updateValue() passes by value — change has no effect | Logical | void updateValue(int *x) { *x = 500; } |
| 8 | 157 | printArray() loop i<=size causes out-of-bounds access | Runtime | for(int i=0; i<size; i++) |
| 8 | 162 | recursiveDemo() has infinite recursion (no base case) | Runtime | Add base case: if(n <= 0) return 0; |
| 8 | 170 | rectangleArea() returns sum instead of product | Logical | return length * width; |
| 8 | 175 | cube() returns n*n (square) instead of n*n*n | Logical | return n * n * n; |
| 9 | 12 | Reading uninitialized pointer ptr1 — undefined behavior | Runtime | Initialize: int val = 0; int *ptr1 = &val; |
| 9 | 16 | Dereferencing NULL pointer ptr2 | Runtime | ptr2 = malloc(sizeof(int)); before *ptr2 = 100; |
| 9 | 22 | Incrementing ptr3 past x — may access garbage memory | Runtime | Do not increment ptr3 past known memory |
| 9 | 27 | Out-of-bounds: *(ptr4+10) on array of size 5 | Runtime | Access only indices 0–4 |
| 9 | 34 | Use-after-free: reading *ptr5 after free(ptr5) | Runtime | Do not access ptr5 after free() |
| 9 | 39 | Memory leak: ptr6 set to NULL without freeing allocated memory | Runtime | free(ptr6); before ptr6 = NULL; |
| 9 | 43 | Heap overflow: writing 10 ints into malloc(5 bytes) | Runtime | ptr7 = malloc(10 * sizeof(int)); |
| 9 | 49 | Out-of-bounds: ptr8[3] on 3-element array (valid: 0–2) | Runtime | Remove ptr8[3] = 40; |
| 9 | 55 | Double free of ptr9 | Runtime | Remove the second free(ptr9); |
| 9 | 59 | Reading uninitialized heap memory from ptr10 | Runtime | Initialize after malloc: *ptr10 = 0; |
| 9 | 64 | Off-by-one: i<=5 writes numbers[5] out of bounds | Runtime | for(int i=0; i<5; i++) |
| 9 | 72 | pptr assigned address of int, but pptr is int** — type mismatch | Semantic | int *valptr = &value; int **pptr = &valptr; |
| 9 | 77 | Buffer overflow: strcpy 'Programming'(11) into malloc(5) | Runtime | name = malloc(12); |
| 9 | 82 | Incrementing ptr11 after malloc — lost track of original pointer | Runtime | Do not modify ptr11 after malloc; use index instead |
| 9 | 86 | Dereferencing NULL pointer ptr12 in if condition | Runtime | Check: if(ptr12 != NULL && *ptr12 > 0) |
| 9 | 92 | Out-of-bounds: ptr13[100] on 100-element array (valid: 0–99) | Runtime | Remove ptr13[100] = 500; |
| 9 | 97 | Use-after-free: writing to ptr14 after free(ptr14) | Runtime | Do not write to ptr14 after free() |

| | | | | |
|---|---|---|---|---|
| 9 | 101 | p assigned matrix (int(*)[3]) but p is int* — type mismatch | Semantic | int *p = &matrix[0][0]; |
| 9 | 106 | Out-of-bounds: *(p+20) on 3x3 matrix (9 elements total) | Runtime | Access only indices 0–8 |
| 9 | 110 | Memory leak: ptr15 first malloc overwritten without freeing | Runtime | free(ptr15); before second malloc |
| 9 | 116 | swap() swaps local pointers, not the values pointed to | Logical | Use temp value: int temp=*a; *a=*b; *b=temp; |
| 9 | 121 | printArray() loop i<=size goes out of bounds | Runtime | for(int i=0; i<size; i++) |
| 10 | 12 | Missing semicolon after 'int choice = 1' | Syntax | int choice = 1; |
| 10 | 14 | Assigning string literal to int variable 'marks' | Semantic | int marks = 90; |
| 10 | 20 | Calling add() with 1 argument instead of 2 | Semantic | result = add(a, b); |
| 10 | 24 | Assignment instead of comparison (a = b) | Semantic | if(a == b) |
| 10 | 28 | Using undeclared variable 'score' | Semantic | Declare 'int score;' before use |
| 10 | 32 | Array out of bounds: arr[10] for arr[5] | Runtime | Use index 0–4 only |
| 10 | 34 | Negative array index arr[-1] — undefined behavior | Runtime | Use valid non-negative index |
| 10 | 38 | Buffer overflow: 'ProgrammingLanguage'(19) into name[10] | Runtime | char name[20]; or use shorter string |
| 10 | 43 | Direct string assignment to char array (city = "Delhi") | Semantic | strcpy(city, "Delhi"); |
| 10 | 49 | Comparing string with == instead of strcmp() | Logical | if(strcmp(user, "admin") == 0) |
| 10 | 54 | Division by zero (x/y where y=0) | Runtime | Ensure y != 0 before dividing |
| 10 | 57 | Dereferencing NULL pointer ptr1 | Runtime | ptr1 = malloc(sizeof(int)); before *ptr1 = 500; |
| 10 | 61 | Reading uninitialized pointer ptr2 | Runtime | Initialize: int val; int *ptr2 = &val; |
| 10 | 66 | Use-after-free: reading *ptr3 after free(ptr3) | Runtime | Do not access ptr3 after free() |
| 10 | 68 | Double free of ptr3 | Runtime | Remove the second free(ptr3); |
| 10 | 72 | Heap overflow: writing 10 ints into malloc(5 bytes) | Runtime | ptr4 = malloc(10 * sizeof(int)); |
| 10 | 77 | Memory leak: ptr5 first malloc overwritten without freeing | Runtime | free(ptr5); before second malloc call |
| 10 | 81 | Calling average() with 2 args instead of 3 | Semantic | avg = average(70, 80, 90); |
| 10 | 86 | Voting eligibility inverted: age>18 prints Not Eligible | Logical | if(age >= 18) printf("Eligible\n"); else printf("Not Eligible\n"); |
| 10 | 93 | Prime check sets isPrime=1 when factor found (should be 0) | Logical | isPrime = 0; break; |
| 10 | 97 | isPrime message inverted: prints 'Not Prime' when prime | Logical | if(isPrime) printf("Prime\n"); else printf("Not Prime\n"); |
| 10 | 101 | Calling undeclared function 'calculateSalary()' | Semantic | Declare/define calculateSalary() before use |
| 10 | 108 | Missing colon after 'case 1' in switch | Syntax | case 1: |
| 10 | 114 | case 2 has no break — falls through to default | Logical | Add break; after printf("Two\n"); |
| 10 | 118 | Multi-character char literal 'AB' assigned to char grade | Syntax | char grade = 'A'; |
| 10 | 122 | Out-of-bounds: scores[3]=40 for scores[3] (valid: 0–2) | Runtime | Remove scores[3]=40; or resize to scores[4] |
| 10 | 127 | Off-by-one in loop: i<=3 accesses scores[3] out of bounds | Runtime | for(int i=0; i<3; i++) |
| 10 | 133 | Circle area formula uses 2*PI*r^2 (sphere) instead of PI*r^2 | Logical | area = 3.14 * radius * radius; |
| 10 | 137 | Semicolon after while condition causes infinite loop | Logical | Remove ';': while(choice > 0) { |
| 10 | 140 | 'continue' used outside a loop | Syntax | Remove the continue; statement |